כריית טקסט – תרגיל בית מספר 4

בתרגיל זה ניקח את הקוד הנלמד בכיתה (E2) המשווה מודלי סיווג שונים וננסה לשנות את המודלים בהתאם להוראות ולבחון האם השינויים משפרים את המודל.

שאלות:

1. (2) הוסיפו למודל BoW גם bigrams. (בנוסף ל unigrams). מה ה Accuracy שהתקבל עבור המודל? דווח את התוצאה לפלט ההגשה.
2. (2) מה ה Accuracy עבור מודל שנבנה בשיטת BoW עם נרמול TF-IDF? בחן את התוצאות עבור אלגוריתם cv.glmnet. דווח את התוצאה לפלט ההגשה.
3. (2) הוסיפו למודל text2vec גם bigrams (בנוסף ל unigrams). מה ה Accuracy שהתקבל עבור המודל? דווח את התוצאה לפלט ההגשה.
4. (2) מה יהיה ה Accuracy עבור מודל text2vec + other variables? דווחו את התוצאה לפלט ההגשה.
5. (2) איזה מודל מכל המודלים המופיעים בקוד והמודלים שבחנתם בתרגיל זה נותנים את התוצאה הטובה ביותר? דווחו לפלט ההגשה.

אופן ההגשה:

- הגשה דרך אתר למידה.
- יש להגיש קובץ R מתועד וקובץ פלט הגשה בפורמט word/pdf המכיל את התשובות לשורות המסומנות בצהוב.
- אין לכווץ את קבצי ההגשה (zip).
- הגשה בזוגות או בבודדים. רק אחד מבני הזוג צריך לטעון את העבודה.
- יש לציין את שמות המגישים על העבודה.

קובץ פלט לתרגיל מספר 4 מגישים: גנאדי בירפיר (ת.ז: 312741622) ו ניב קוטק (ת.ז: 208236315)

סעיף	מטלה	פתרון
1	Accuracy (BoW, unigram+**bigram**, cv.glmnet)	Accuracy = 0.6626471 = 66.264%
2	Accuracy (test set) with **tf-idf** (BoW, cv.glmnet)	Accuracy = 0.6741176 = 67.411%
3	Accuracy (text2vec, unigram+**bigram**, cv.glmnet)	Accuracy = 0.6594118 = 65.941%
4	Accuracy (test set) (**text2vec**, **combined variables**, cv.glmnet)	Accuracy = 0.7426471 = 74.264%
5	איזה מודל נתן את התוצאה הכי טובה מבין כל המודלים? (המודלים המופיעים בקוד והמודלים שאתם הוספתם)	המודל של **BoW + other variables** נותן את התוצאה **<u>הכי טובה</u>** ומגיע לדיוק של כמעט 74.5%. (בסוף הדף יש טבלת סיכום). (נ.ב: ניתן לראות שהמודל text2vec + other variables נותן תוצאה של כמעט 74.3% אשר כמעט דומה למודל הכי טוב – כלומר שאנחנו מוסיפים other variables ל BoW או ל text2vec התוצאה משתפרת.)

	BoW	**text2vec**	**other variables**	**BoW + other variables**	**text2vec + other variables**	**BoW and TF-IDF**	**text2vec and bigrams**	**BoW and bigrams**
lasso	0.6747059	0.6667647	0.7347059	**0.7444118**	0.7426471	0.6741176	0.6594118	0.6626471

```
# set working directory
setwd("C:/Users/niv/Desktop/Text mining/EX4")

## libraries
library(tm)
library(text2vec) # text2vec representation
library(glmnet) # lasso regression
library(caret) # confusionMatrix
library(e1071) # SVM
library(qdap) # corpus to data.frame
library(RWeka)

## read text
options(stringsAsFactors = F) # make sure strings are read as strings, and not factors
diabetesDF <- read.csv("diabetes_subset_8500.csv", encoding="UTF-8")
diabetesDF$diag.text <- paste(diabetesDF$diag_1_desc,
                              diabetesDF$diag_2_desc,
                              diabetesDF$diag_3_desc, sep=" ")
```

```
diabetesDF$diag.text <- iconv(diabetesDF$diag.text, "UTF-8", "ASCII", sub="")


## pre-process
preprocess <- function(text){
  myCorpus <- VCorpus(VectorSource(text))
  myCorpus <- tm_map(myCorpus, content_transformer(tolower))
  myCorpus <- tm_map(myCorpus, removeWords, stopwords("english"))
  myCorpus <- tm_map(myCorpus, removeNumbers)
  myCorpus <- tm_map(myCorpus, removePunctuation)
  myCorpus <- tm_map(myCorpus, stripWhitespace)
  myCorpus <- tm_map(myCorpus, stemDocument)
  return(myCorpus)
}


## partition
set.seed(1)
trainIDs<-createDataPartition(diabetesDF$readmitted,p=0.6,list = F) # p -> the percentage of data that goes to training. #list = F -> the result will be in a matrix and not a list
```

```
#createDataPartition attempts to balance the class distributions within the splits.
train.diabetes<-diabetesDF[trainIDs,]
test.diabetes<-diabetesDF[-trainIDs,]

#preprocess the training set
diabetesCorpus.train <- preprocess(train.diabetes$diag.text)
for(i in 1:5){
  print(diabetesCorpus.train[[i]][1])
}

# functions ----------cv.glmnet---------
lassoPredict <- function(train.mat, test.mat){

  cv <- cv.glmnet(train.mat,
                  y = as.factor(train.diabetes$readmitted),
                  alpha=0.1,
                  family='binomial',
                  nfolds=10,
```

```
                intercept=F,
                type.measure = 'class')

  preds <- predict(cv, test.mat,
                 type = 'class', s=cv$lambda.1se)

  cm <- confusionMatrix(as.factor(preds),
                      as.factor(test.diabetes$readmitted))

  return(cm$overall[1])

}

#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~Q.1~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
# BoW + Bigram Tokenizer
BigramTokenizer <- function(x) NGramTokenizer(x, Weka_control(min = 1, max = 2)) #define the function BigramTokenizer
train.bigram.dtm <- DocumentTermMatrix(diabetesCorpus.train, control = list(tokenize = BigramTokenizer))
#BigramTokenizer function is passed as a parameter
```

```
#inspect(train.bigram.dtm)


#preprocess the test set

diabetesCorpus.test <- preprocess(test.diabetes$diag.text)


#create a test DTM

test.bigram.dtm <- DocumentTermMatrix(diabetesCorpus.test, control = list(dictionary=Terms(train.bigram.dtm),
tokenize=BigramTokenizer))

#inspect(test.bigram.dtm)


# lasso regression: y ~ BoW

train.bigram.matrix<-Matrix(as.matrix(train.bigram.dtm), sparse = T)

test.bigram.matrix<-Matrix(as.matrix(test.bigram.dtm), sparse = T)

set.seed(1)

lassoPredict(train.bigram.matrix, test.bigram.matrix) # Accuracy  = 0.6626471 = 66.264%


#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~Q.2~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~

# BoW + TF-IDF
```

```
train.TFIDF.dtm <- DocumentTermMatrix(diabetesCorpus.train,control=list(weighting=weightTfIdf)) #weightTfIdf weights a DTM by TF-IDF

#inspect(train.TFIDF.dtm)


#preprocess the test set

diabetesCorpus.test <- preprocess(test.diabetes$diag.text)


#create a test DTM

test.TFIDF.dtm <- DocumentTermMatrix(diabetesCorpus.test, control = list(dictionary=Terms(train.TFIDF.dtm), weighting=weightTfIdf))  #weightTfIdf weights a DTM by TF-IDF

#inspect(test.dtm)


# lasso regression: y ~ BoW

train.TFIDF.matrix<-Matrix(as.matrix(train.TFIDF.dtm), sparse = T)

test.TFIDF.matrix<-Matrix(as.matrix(test.TFIDF.dtm), sparse = T)

set.seed(1)

lassoPredict(train.TFIDF.matrix, test.TFIDF.matrix) # Accuracy  = 0.6741176  = 67.411%


#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~Q.3~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
```

```
# Text2Vec + bigrams

train.diabetes$diag.text <- as.data.frame(diabetesCorpus.train)[,2] # change original text to the cleaned text (for text2vec)

test.diabetes$diag.text <- as.data.frame(diabetesCorpus.test)[,2] #change original text to the cleaned text (for text2vec)

it <- itoken(train.diabetes$diag.text) # Creates iterators over input objects in order to create vocabularies

v.bigrams <- create_vocabulary(it,ngram = c(1, 2)) # This function collects unique terms and corresponding statistics (add ngram)

print(v.bigrams)

vectorizer.bigrams <- vocab_vectorizer(v.bigrams) # Defines on how to transform list of tokens into vector space

tcm <- create_tcm(it, vectorizer.bigrams, skip_grams_window = 5L)

glove <- GlobalVectors$new(rank = 100, x_max = 10)#x_max = maximum number of co-occurrences to use in the weighting function

word_context <- glove$fit_transform(tcm, n_iter = 100)

word_vectors <- glove$components + t(word_context)

word_vectors[1:100,1:10]

dim(word_vectors)

train.bigrams.dtm <- create_dtm(it, vectorizer.bigrams)
```

```
dim(train.bigrams.dtm)

train.bigrams.dtm <- as.matrix((train.bigrams.dtm/Matrix::rowSums(train.bigrams.dtm)) %*% t(word_vectors)) # doc2vec - calculate an average vector of all the words


test.it <- itoken(test.diabetes$diag.text)

test.bigrams.dtm <- create_dtm(test.it, vectorizer.bigrams) # vectorizer contains train vocabulary

test.bigrams.dtm <- as.matrix((test.bigrams.dtm/Matrix::rowSums(test.bigrams.dtm)) %*% t(word_vectors))

dim(test.bigrams.dtm)

# lasso regression: y ~ text2vec

set.seed(1)

lassoPredict(train.bigrams.dtm,test.bigrams.dtm) # Accuracy  = 0.6594118   = 65.941%


#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~Q.4~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~

# Text2Vec + other variables


train.diabetes$diag.text <- as.data.frame(diabetesCorpus.train)[,2] # change original text to the cleaned text (for text2vec)

test.diabetes$diag.text <- as.data.frame(diabetesCorpus.test)[,2] #change original text to the cleaned text (for text2vec)


it <- itoken(train.diabetes$diag.text) # Creates iterators over input objects in order to create vocabularies
```

```
v <- create_vocabulary(it) # This function collects unique terms and corresponding statistics

print(v)

vectorizer <- vocab_vectorizer(v) # Defines on how to transform list of tokens into vector space


tcm <- create_tcm(it, vectorizer, skip_grams_window = 5L)

glove <- GlobalVectors$new(rank = 100, x_max = 10)#x_max = maximum number of co-occurrences to use in the weighting function

word_context <- glove$fit_transform(tcm, n_iter = 100)

word_vectors <- glove$components + t(word_context)

word_vectors[1:100,1:10]

dim(word_vectors)


train.dtm <- create_dtm(it, vectorizer)

dim(train.dtm)

train.dtm <- as.matrix((train.dtm/Matrix::rowSums(train.dtm)) %*% t(word_vectors)) # doc2vec - calculate an average vector of all the words


test.it <- itoken(test.diabetes$diag.text)

test.dtm <- create_dtm(test.it, vectorizer) # vectorizer contains train vocabulary
```

```
test.dtm <- as.matrix((test.dtm/Matrix::rowSums(test.dtm)) %*% t(word_vectors))

dim(test.dtm)

# lasso regression: y ~ text2vec

set.seed(1)

## Text2Vec + other variables

lassoPredict(cbind(train.dtm, as.matrix(train.diabetes[,1:132])),

        cbind(test.dtm, as.matrix(test.diabetes[,1:132]))) # Accuracy  = 0.7426471      = 74.263%


#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~Q.5~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~

# other


#create a train DTM

train.dtm <- DocumentTermMatrix(diabetesCorpus.train)

# inspect(train.dtm)


#preprocess the test set

diabetesCorpus.test <- preprocess(test.diabetes$diag.text)
```

```
#create a test DTM
test.dtm <- DocumentTermMatrix(diabetesCorpus.test, control = list(dictionary=Terms(train.dtm)))
# inspect(test.dtm)

# BoW ----------------------------------------------------------------

# lasso regression: y ~ BoW
train.matrix<-Matrix(as.matrix(train.dtm), sparse = T)
test.matrix<-Matrix(as.matrix(test.dtm), sparse = T)
#3.todo - try not to convert to a Matrix, what is the difference?
set.seed(1)
lassoPredict(train.matrix, test.matrix) # Accuracy = 0.6747059   =67.470%

# other variables ----------------------------------------------------
# lasso regression: y ~ numeric variables
set.seed(1)
lassoPredict(as.matrix(train.diabetes[,1:132]), as.matrix(test.diabetes[,1:132])) # Accuracy = 0.7347059   =73.47%
```

```
# combine attributes ------------------------------------------------

set.seed(1)

lassoPredict(cbind(train.matrix, as.matrix(train.diabetes[,1:132])),

        cbind(test.matrix, as.matrix(test.diabetes[,1:132]))) # Accuracy = 0.7444118     =74.441%


# text2vec ----------------------------------------------------

train.diabetes$diag.text <- as.data.frame(diabetesCorpus.train)[,2] # change original text to the cleaned text (for text2vec)

test.diabetes$diag.text <- as.data.frame(diabetesCorpus.test)[,2] #change original text to the cleaned text (for text2vec)


##

it <- itoken(train.diabetes$diag.text) # Creates iterators over input objects in order to create vocabularies

v <- create_vocabulary(it) # This function collects unique terms and corresponding statistics

vectorizer <- vocab_vectorizer(v) # Defines on how to transform list of tokens into vector space


tcm <- create_tcm(it, vectorizer, skip_grams_window = 5L)

glove <- GlobalVectors$new(rank = 100, x_max = 10)#x_max = maximum number of co-occurrences to use in the weighting function

word_context <- glove$fit_transform(tcm, n_iter = 100)
```

```
word_vectors <- glove$components + t(word_context)

word_vectors[1:100,1:10]

dim(word_vectors)


train.dtm <- create_dtm(it, vectorizer)

dim(train.dtm)

train.dtm <- as.matrix((train.dtm/Matrix::rowSums(train.dtm)) %*% t(word_vectors)) # doc2vec - calculate an average vector of all the words


test.it <- itoken(test.diabetes$diag.text)

test.dtm <- create_dtm(test.it, vectorizer) # vectorizer contains train vocabulary

test.dtm <- as.matrix((test.dtm/Matrix::rowSums(test.dtm)) %*% t(word_vectors))

dim(test.dtm)

# lasso regression: y ~ text2vec

set.seed(1)

lassoPredict(train.dtm,test.dtm) # Accuracy = 0.6667647   =66.676%
```